

TaskFlow → Shop & Inventory Management System

Project blueprint, technology stack, MongoDB design, roles, APIs, and implementation roadmap

1. Project goal

Convert the existing TaskFlow authentication and profile-picture project into a shop-management and inventory-management REST API using JWT authentication and MongoDB. Super Admin controls accounts and system-wide access; Admin users manage inventory; Managers handle customers and sales.

2. User roles and permissions

Role	Responsibility	Permissions
Super Admin	System owner	Manage users, roles, settings, inventory, customers, reports, and account status.
Admin	Inventory operator	Manage companies, items, quantities, cost price, selling price, stock adjustments, and inventory re
Manager	Customer and sales operator	Manage customers, sales, invoices/orders, payments, and permitted stock information.

3. Technology stack

- Backend: Python with FastAPI.
- Database: MongoDB using Motor for asynchronous access.
- Authentication: JWT access tokens using python-jose.
- Password security: bcrypt hashing with a compatible version.
- Validation: Pydantic models and EmailStr.
- File storage: local uploads directory initially; object storage can be added later.
- Testing: Swagger UI at /docs and ReDoc at /redoc.
- Database administration: MongoDB Compass.

4. Project structure

```
taskflow/  
■■■■ main.py  
■■■■ app/  
■ ■■■■ database.py  
■ ■■■■ config.py  
■ ■■■■ models.py  
■ ■■■■ auth.py  
■ ■■■■ deps.py  
■ ■■■■ roles.py  
■ ■■■■ users.py  
■ ■■■■ inventory.py  
■ ■■■■ companies.py  
■ ■■■■ customers.py  
■ ■■■■ sales.py  
■ ■■■■ reports.py  
■■■■ uploads/profile_images/  
■■■■ requirements.txt
```

5. MongoDB design

Use a database named **shop_management**. Store created_at and updated_at timestamps where appropriate.

Collection	Important fields
users	_id, name, email, password_hash, role, profile_image, is_active, created_at, updated_at
companies	_id, company_name, phone, email, address, created_at

Collection	Important fields
items	_id, item_id, item_name, company_id, quantity, cost_price, selling_price, reorder_level, unit, is_active, created_at, updated_at
stock_movements	_id, item_id, type, quantity, reference, performed_by, created_at
customers	_id, customer_code, name, phone, email, address, opening_balance, created_at, updated_at
sales	_id, invoice_no, customer_id, items, subtotal, discount, tax, total, paid_amount, due_amount, status, created_by, created_at
payments	_id, customer_id, sale_id, amount, method, received_by, created_at

6. Item fields

- `item_id`: unique business identifier, such as ITM-0001.
- `item_name`: product name.
- `company_id`: supplier/manufacturer reference.
- `quantity`: available stock.
- `cost_price` (CP): purchase cost per unit.
- `selling_price` (SP): selling price per unit.
- `reorder_level` and `unit`: low-stock threshold and measurement unit.
- `created_at` and `updated_at`: audit timestamps.

Use stock-movement records for purchases, sales, returns, and manual adjustments instead of silently changing quantity.

7. Authentication and authorization

- User signs up or is created by Super Admin.
- Passwords are hashed; plain-text passwords are never stored.
- Login returns a JWT containing user ID and role.
- Protected routes use `get_current_user`.
- Role dependencies enforce access, such as `require_role('super_admin')` or `require_role('admin', 'super_admin')`.
- Inactive users are rejected.
- Use short-lived access tokens; add refresh tokens later if needed.

8. Core API plan

Method	Endpoint	Access	Purpose
POST	<code>/auth/signup</code>	Public/Super Admin	Create account
POST	<code>/auth/login</code>	Public	Login and receive JWT
GET	<code>/users</code>	Super Admin	List users
PUT	<code>/users/{id}/role</code>	Super Admin	Change role
PATCH	<code>/users/{id}/status</code>	Super Admin	Activate/deactivate
GET	<code>/profile/me</code>	Logged in	View own profile
PUT	<code>/profile/me</code>	Logged in	Update own profile
POST	<code>/profile/upload</code>	Logged in	Upload/replace picture
DELETE	<code>/profile/image</code>	Logged in	Delete picture
POST	<code>/companies</code>	Admin/Super Admin	Create company
GET	<code>/companies</code>	Logged in	List companies
POST	<code>/items</code>	Admin/Super Admin	Create item
GET	<code>/items</code>	Logged in	Search/list items
PUT	<code>/items/{id}</code>	Admin/Super Admin	Update item
DELETE	<code>/items/{id}</code>	Admin/Super Admin	Soft-delete item
POST	<code>/items/{id}/stock-in</code>	Admin/Super Admin	Add stock
POST	<code>/items/{id}/stock-out</code>	Admin/Manager	Remove stock
POST	<code>/customers</code>	Manager/Super Admin	Create customer

Method	Endpoint	Access	Purpose
GET	/customers	Manager/Super Admin	List/search customers
PUT	/customers/{id}	Manager/Super Admin	Update customer
DELETE	/customers/{id}	Manager/Super Admin	Deactivate customer
POST	/sales	Manager/Super Admin	Create sale and reduce stock
GET	/sales	Manager/Super Admin	List sales
POST	/payments	Manager/Super Admin	Record payment

9. Business rules

- item_id must be unique.
- CP and SP must be non-negative.
- Stock should not become negative unless explicitly allowed.
- Every stock change creates a stock_movements document.
- Sales should validate stock before reducing it.
- Use MongoDB transactions when running a replica set.
- Use soft delete where historical sales must remain readable.
- Never expose password_hash in API responses.
- Add indexes on email, item_id, company_id, customer code/phone, invoice_no, and created_at.

10. Implementation order

- Keep and test existing authentication and profile features.
- Add role field and create the first Super Admin.
- Add role-checking dependencies.
- Build companies CRUD.
- Build items CRUD with item_id, CP, SP, and quantity.
- Add stock-in, stock-out, and movement history.
- Build customers CRUD.
- Build sales and payments with stock validation.
- Add low-stock, inventory-value, sales, and outstanding-balance reports.
- Add frontend dashboard, tests, environment variables, backups, and deployment.

11. Example item document

```
{
  "item_id": "ITM-0001",
  "item_name": "Wireless Mouse",
  "company_id": "ObjectId(...)",
  "quantity": 50,
  "cost_price": 450.00,
  "selling_price": 650.00,
  "reorder_level": 10,
  "unit": "piece",
  "is_active": true
}
```

12. Required packages

FastAPI, Uvicorn, Motor, PyMongo, python-multipart, python-jose, passlib with a compatible bcrypt version, and email-validator. Pin versions after testing in the project virtual environment.

Final architecture: Super Admin manages the platform and accounts; Admin manages companies, items, CP, SP, and stock; Manager manages customers, sales, and payments. MongoDB stores all business data, and JWT plus role dependencies protect each API.